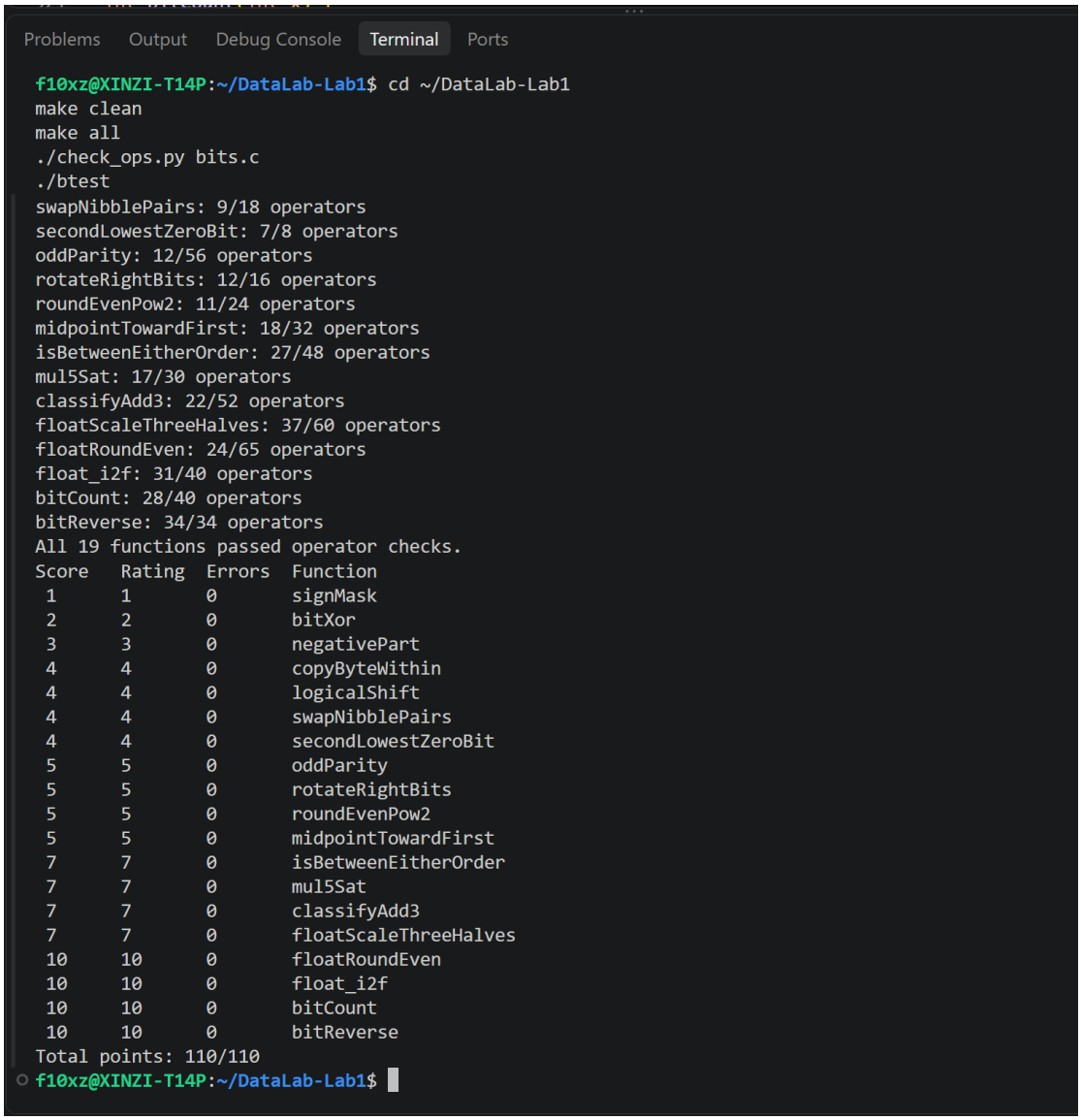
# Lab1:Datalab 实验报告

夏玉新子 25800440036

## 一. 检验结果：




## 二. 实现思路：

### 1. SignMask

本题要求返回一个只有最高位为 1、其余位均为 0 的 32 位掩码。由于不能直接使用超过 255 的整数常量，我将整数 1 左移 31 位，得到所需的最高位掩码。实现代码如下：

```
// P1
/*
 * signMask - return a mask with only the most significant bit set (0x80000000)
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 2
 *   Rating: 1
 */
int signMask(void) {
  return 1<<31;
}
```

## 2. bitXor

本题要求只使用按位取反~和按位与&，实现两个整数的按位异或：两个对应位相同时应输出 0，不同时应输出 1。该条件等价于：若输出 1，则至少有一位为 1，并且不能两位都为 1。

对于“至少有一位为 1”，我使用~(~x &~y)，先分别对两位取反，再按位与，只有原来的两位都是 0 时才得到 1；将结果再次取反，就使 00 输出 0，其余情况输出 1。

对于“不能两位都为 1”，我使用~(x & y)。按位与只有在两位都是 1 时才得到 1，取反后就使 11 输出 0，其余情况输出 1。

最后，将两个条件按位与。对于 00，第一个条件为 0；对于 11，第二个条件为 0；对于 01 和 10，两个条件都为 1。由此，实现了两个整数的按位异或。实现代码如下：

```
// P2
/*
 * bitXor - x^y using only ~ and &
 *   Example: bitXor(4, 5) = 1, bitXor(7, 7) = 0
 *   Legal ops: ~ &
 *   Max ops: 8
 *   Rating: 2
 */
int bitXor(int x, int y) {
  return ~(x & y) & ~(~x & ~y);
}
```

## 3. negativePart

本题要求负数返回相反数，非负数返回 0。由于不能使用 if 或比较运算符，需要通过位运算实现。

首先，计算 x>>31。右移 31 位后，如果 x 为负数，结果的全部位都是 1；如果 x 为非负数，全部位都是 0。由此得到一个由输入符号决定的掩码。

然后，通过~x+1 计算其相反数。

最后，将相反数与符号掩码按位与。当 x 为负数时，掩码全为 1，按位与会保留相反数的全部位。当 x 为非负数时，掩码全为 0，按位与会清除全部位，因此返回 0。实现代码如下：

```
// P3
/*
 * negativePart - return -x if x < 0, otherwise return 0
 *   Examples: negativePart(-10) = 10, negativePart(5) = 0
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 6
 *   Rating: 3
 */
int negativePart(int x){
  return (x >> 31) & (~x + 1);
}
```

**4. copyByteWithin**

题目要求将 x 中编号为 src 的字节复制到编号为 dst 的字节，其他字节保持不变。

由于每个字节占 8 位，字节编号乘以 8 就是它的起始位位置。由于不允许使用乘法，我通过将字节编号左移 3 位，计算源字节和目标字节的位置。

首先，取出源字节：将 x 右移 srcShift 位，使源字节落到最低 8 位，再与 255 按位与。255 的低 8 位全为 1，其他位全为 0，因此能够保留源字节，同时清除其他位。

接着，清除目标字节的原内容：将 255 左移到目标字节的位置，得到一个只有目标字节的 8 位为 1 的掩码。对该掩码取反后，目标字节位置变为全 0，其他位置变为全 1。

最后，将取出的源字节左移 dstShift 位，放到目标位置，再与清空目标字节后的整数按位或，从而实现字节的复制。实现代码如下：

```
// P4
/*
 * copyByteWithin - copy byte src of x to byte dst, leaving all other bytes unchanged
 *   Bytes are numbered from 0 (least significant) to 3 (most significant).
 *   You can assume 0 <= src <= 3 and 0 <= dst <= 3.
 *   Example: copyByteWithin(0x11223344, 0, 2) = 0x11443344
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 12
 *   Rating: 4
 */
int copyByteWithin(int x, int src, int dst) {
  int srcShift = src << 3;
  int dstShift = dst << 3;
  int byte = (x >> srcShift) & 255;
  int mask = 255 << dstShift;
  return (x & ~mask) | (byte << dstShift);
}
```

## 5. logicalShift

题目要求将整数逻辑右移，即右移后高位补 0。但实验规定，有符号整数的右移是算术右移：非负数补 0，负数补 1。因此，我先对整数进行算术右移，再用掩码清除高位补入的符号位。

右移 n 位后，原整数中需要保留的内容位于低 32−n 位。因此，所需掩码应当是高 n 位全为 0、低 32−n 位全为 1。将右移结果与这个掩码按位与，就能清除高位，同时保留其余内容。

为构造掩码，先将整数 1 左移 31 位，得到只有最高位为 1 的位模式。将它算术右移 n 位后，高位补入 1，使最高的 n＋1 位都为 1。再左移一位，原来的最高位被丢弃，得到高 n 位为 1、其余位为 0 的结果（为使 n=1 时也成立）。最后将所有位取反，就得到所需掩码。实现代码如下：

```
// P5
/*
 * logicalShift - shift x to the right by n bits, using a logical shift
 *   Can assume that 0 <= n <= 31
 *   Examples: logicalShift(0x87654321,4) = 0x08765432
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 20
 *   Rating: 4
 */
int logicalShift(int x, int n) {
  int mask = ~(((1 << 31) >> n) << 1);
  return (x >> n) & mask;
}
```

### 6. swapNibblePairs

题目要求交换每个字节内部的高四位和低四位。为实现题目要求，我分别提取两组位，将原来的低四位移到高四位，将原来的高四位移到低四位，最后合并。

首先，构造掩码 0x0F0F0F0F。将 15 左移 8 位，再与原来的 15 按位或，得到 0x00000F0F；将这个结果左移 16 位，再与未移动的结果按位或，得到完整掩码。该掩码在每个字节的低四位为 1、高四位为 0。

将原整数与该掩码按位与，然后将结果左移 4 位，使每个字节原来的低四位进入同一字节的高四位。由于提取结果中各字节的高四位原本都是 0，这次左移不会把有效内容移入相邻字节。

将原整数右移 4 位。右移后，每个字节原来的高四位进入同一字节的低四位。随后将右移结果与该掩码按位与，只保留各字节低四位中的目标内容，清除高四位中的无关内容。

最后，将两部分按位或，得到交换结果。

```
// P6
/*
 * swapNibblePairs - swap the low and high 4 bits within each byte of x
 *   Examples: swapNibblePairs(0xAB) = 0xBA
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 18
 *   Rating: 4
 */
int swapNibblePairs(int x) {
  int mask = 15 | (15 << 8);
  mask = mask | (mask << 16);
  return ((x & mask) << 4) | ((x >> 4) & mask);
}
```

### 7. secondLowestZeroBit

题目要求从最低位向最高位寻找第二个为 0 的位，并返回只有该位置为 1、其余位置为 0 的掩码。如果整数中不足两个 0，则返回 0。

首先，对原整数按位取反。接着，先计算取反结果减一后的值，再将其与取反结果按位与。由于题目禁止减法，通过加上全 1 的补码实现减一。最后，从剩余结果中提取最低的 1：对剩余结果取反加 1，得到其相反数的补码表示，再与剩余结果按位与。按位与后只有最低的 1 被保留，更高的位全部被清除。这个位置就是原整数中第二个 0 的位置。

```
// P7
/*
 * secondLowestZeroBit - return a mask that marks the position of the second least significant 0 bit
 *   Examples: secondLowestZeroBit(0xFFFFFFFA) = 0x4, secondLowestZeroBit(0x7FFFFFFF) = 0
 *             secondLowestZeroBit(-1) = 0
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 8
 *   Rating: 4
 */
int secondLowestZeroBit(int x) {
  int zeros = ~x;
  int rest = zeros & (zeros + ~0);
  return rest & (~rest + 1);
}
```

### 8. oddParity

题目要求判断整数的 32 位二进制表示中，1 的数量是奇数还是偶数：数量为偶数时返回 1，数量为奇数时返回 0。因此，不需要计算 1 的具体数量，只需要得到数量的奇偶性。

利用异或运算判断奇偶性。两个二进制位异或时，两位相同得到 0，两位不同得到 1。将多个二进制位连续异或，相当于每遇到一个 1 就将结果翻转一次：遇到偶数个 1，结果为 0；遇到奇数个 1，结果为 1。

将整数与右移 16 位后的整数异或，再将当前结果与右移 8 位后的结果异或，随后在右移 4 位、2 位和 1 位后继续分别异或。异或完成后，用最低位为 1、其他位为 0 的掩码提取最低位。最低位为 0 表示原整数中有偶数个 1，为 1 表示有奇数个 1。由于题目要求偶数时返回 1、奇数时返回 0，再对提取结果取逻辑非，使返回值符合要求。实现代码如下：

```
// P8
/*
 * oddParity - return the odd parity bit of x, that is,
 *      when the number of 1s in the binary representation of x is even, then the return 1, otherwise return 0.
 *   Examples: oddParity(5) = 1, oddParity(7) = 0
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 56
 *   Rating: 5
 */
int oddParity(int x) {
  x = x ^ (x >> 16);
  x = x ^ (x >> 8);
  x = x ^ (x >> 4);
  x = x ^ (x >> 2);
  x = x ^ (x >> 1);
  return !(x & 1);
}
```

### 9. rotateRightBits

题目要求将整数循环右移 n 位。由于普通右移会丢弃移出的低位，需要分别计算右移后保留的部分和需要移到高位的部分，再将两部分合并。

首先，由于每循环右移 32 位都会恢复原来的位序，只需要考虑 n 除以 32 的余数。由于不允许使用取模运算，将 n 与 31 按位与，并保留 n 的低五位。

对于右移后保留的部分，先将原整数算术右移，再用掩码清除高位补入的符号位。该掩码的高位为 0，需要保留的低位为 1。这样得到逻辑右移的结果，其高位空出的位置用于接收移出的低位。

对于需要移到高位的部分，由于右移位数可能为 0，先通过取反加 1 得到实际旋转量的相反数，再与 31 按位与。随后，将原整数左移“32 减去实际右移位数”位。

最后，将两部分按位或，得到最终结果。实现代码如下。

```
// P9
/*
 * rotateRightBits - rotate x to right by n bits
 *   you can assume n >= 0
 *   Examples: rotateRightBits(0x12345678, 8) = 0x78123456
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 16
 *   Rating: 5
 */
int rotateRightBits(int x, int n) {
  int right = n & 31;
  int left = (~right + 1) & 31;
  int mask = ~(((1 << 31) >> right) << 1);
  return ((x >> right) & mask) | (x << left);
}
```

**10. roundEvenPow2**

题目要求将非负整数舍入到最近的 2 的 n 次方的倍数。如果输入恰好位于两个相邻倍数的中间，就选择除以该倍数单位后，商为偶数的结果。

先将输入右移 n 位，得到输入除以 2 的 n 次方后向下取整的商。

为了不用条件语句完成选择，在输入上加一个数，再右移 n 位。这个要加的数由商的奇偶性决定：商为偶数时，加上“半个单位减一”；商为奇数时，加上“半个单位”。商的奇偶用其最低位区分。若加上这个数后，余数达到一个完整单位，右移得到的商就会增加 1；若余数仍不足一个单位，商就保持不变。

最后，将调整后的商左移 n 位，得到对应的倍数。

```c
// P10
/*
 * roundEvenPow2 - round nonnegative x to the nearest multiple of 2^n.
 *   If x is exactly halfway between two multiples, choose the multiple whose
 *   quotient by 2^n is even.
 *   You can assume 0 <= x <= 0x3fffffff and 1 <= n <= 16.
 *   Examples: roundEvenPow2(10, 2) = 8, roundEvenPow2(14, 2) = 16
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 24
 *   Rating: 5
 */
int roundEvenPow2(int x, int n) {
  int quotient = x >> n;
  int half = 1 << (n + ~0);
  int bias = half + ~0 + (quotient & 1);
  return ((x + bias) >> n) << n;
}
```

**11. midpointTowardFirst**

题目要求计算两个整数的数学中点，并避免计算过程溢出。如果中点不是整数，就选择靠近第一个参数的那个整数。

直接将两个整数相加可能溢出，因此通过按位与和异或拆分计算。按位与保留两数对应位都为 1 的部分，异或保留对应位不同的部分。两数相加时，共同部分需要计算两次，不同部分只计算一次；求平均时，就可以保留共同部分，再加上不同部分的一半。

因此，先计算两数的按位与结果，再将两数的异或结果算术右移一位，相当于除以 2 后向下取整，最后将这两部分相加。

接着，判断中点是否为整数。两个整数相加得到奇数，说明它们的最低位不同；相加得到偶数，说明最低位相同。因此，只需要检查两数异或结果的最低位：最低位为 1，说明中点位于两个相邻整数之间；最低位为 0，说明中点已经是整数。

当中点不是整数时，调整方向取决于第一个参数与第二个参数的大小。如果第一个参数较小，向下取整得到的整数更靠近第一个参数，可以直接保留；如果第一个参数较大，较大的相邻整数更靠近第一个参数，需要将当前结果加一。

因此，还需要判断两个参数的大小。两数异号时，非负数一定大于负数，可以直接根据符号判断；两数同号时，通过“第二个参数减去第一个参数”的结果是否为负来判断大小，减法则通过取反加一和加法实现。

当“中点不是整数”和“第一个参数较大”两个条件同时满足时，将向下取整的中点加一，其余情况保持不变。实现代码如下：

```
// P11
/*
 * midpointTowardFirst - return the exact mathematical midpoint (x+y)/2
 *   without overflow. If the exact midpoint lies halfway between two
 *   integers, choose the adjacent integer that is closer to the first
 *   argument x.
 *   Examples: midpointTowardFirst(4, 7) = 5,
 *             midpointTowardFirst(7, 4) = 6
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 32
 *   Rating: 5
 */
int midpointTowardFirst(int x, int y) {
  int differing = x ^ y;
  int average = (x & y) + (differing >> 1);
  int signDiff = differing >> 31;
  int xGreater = (signDiff & ~(x >> 31)) |
                 (~signDiff & ((y + ~x + 1) >> 31));
  return average + (differing & xGreater & 1);
}
```

### 12. isBetweenEitherOrder

题目要求判断整数是否位于两个端点构成的闭区间内。

首先，判断输入是否小于第一个端点。直接根据减法结果的符号判断大小。由于可能因整数溢出而出错，需要先区分两数是否同号。对输入与端点进行异或，并检查异或结果的符号位：符号位不同说明两数异号，符号位相同说明两数同号。

两数异号时，可以直接根据输入的符号判断大小。两数同号时，可以根据差值是否为负判断大小。减法通过对端点取反加一，再与输入相加来实现。随后，用相同的方法判断输入是否小于第二个端点。

接下来，比较这两个判断结果。将两次结果异或并提取最低位，就能判断输入是否位于这一闭区间内。

另外，还需要考虑输入等于任一端点的情况。两个整数相等时，异或结果为 0；对该结果取逻辑非，就能在相等时得到 1，不相等时得到 0。将区间判断结果与两次相等判断结果按位或，使输入等于端点时也返回 1。具体实现代码如下：

```c
// P12
/*
 * isBetweenEitherOrder - return 1 when x lies in the inclusive interval whose
 *   endpoints are a and b. The endpoints may be given in either order.
 *   Example: isBetweenEitherOrder(5, 8, 3) = 1.
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 48
 *   Rating: 7
 */
int isBetweenEitherOrder(int x, int a, int b) {
  int ax = x ^ a;
  int bx = x ^ b;
  int signA = ax >> 31;
  int signB = bx >> 31;
  int signX = x >> 31;
  int lessA = (signA & signX) |
              (~signA & ((x + ~a + 1) >> 31));
  int lessB = (signB & signX) |
              (~signB & ((x + ~b + 1) >> 31));
  return ((lessA ^ lessB) & 1) | !ax | !bx;
}
```

### 13. mul5Sat

题目要求计算输入整数的五倍。如果数学结果超过有符号 32 位整数的上限，就返回 INT_MAX；如果低于下限，就返回 INT_MIN。

先通过加法计算五倍，再检查计算过程中是否溢出，最后选择应返回的结果。由于不允许使用乘法，先将输入与自身相加，得到两倍；再将两倍结果与自身相加，得到四倍；最后将四倍结果与原输入相加，得到五倍。

随后，检查每一步的符号变化。若结果与输入的符号不同，就说明已经溢出。使用位运算完成检查：分别对每一步计算前后的数进行异或。符号不同会使异或结果的最高位为 1。将三次异或结果按位或，再算术右移 31 位，就能得到统一的判断结果：发生过溢出时全部位为 1，没有溢出时全部位为 0。

最后，根据原输入的符号构造溢出时应返回的值。具体实现代码如下：

```c
// P13
/*
 * mul5Sat - return x*5, and if x*5 overflow, change the result to
 * INT_MAX(0x7fffffff) or INT_MIN(0x80000000) correspondingly
 *   Examples: mul5Sat(1) = 0x5, mul5Sat(0x40000000) = 0x7fffffff
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 30
 *   Rating: 7
 */
int mul5Sat(int x) {
  int twice = x + x;
  int four = twice + twice;
  int five = four + x;
  int overflow = ((x ^ twice) | (twice ^ four) | (four ^ five)) >> 31;
  int limit = (1 << 31) ^ ~(x >> 31);
  return (overflow & limit) | (~overflow & five);
}
```

### 14. classifyAdd3

题目要求判断三个整数的精确数学和是否超出有符号 32 位整数的范围。由于直接相加可能溢出，我先计算数学总和除以 4 后向下取整的值，再根据这个值判断是否越界。

首先，将每个输入拆成“4 倍的商加余数”。对输入算术右移两位，可以得到除以 4 后向下取整的商；将输入与 3 按位与，可以得到低两位表示的余数。

接着，分别汇总三个商和三个余数。三个商相加后，还需要考虑余数相加产生的完整单位：将余数总和右移两位，得到需要增加的数量，再加到三个商的总和上。这样得到的结果，就是三个输入的精确数学和除以 4 后向下取整的值。

然后，把整数上下限转换为相应的判断界限。上限为 536870912，下限为 −536870912。大于等于上限或小于下限时均溢出。将计算出的商减去正界限，可以判断是否超过上限；将该商加上正界限，可以判断是否低于下限，减法通过取反加一和加法实现。按位或后，即可得到题目要求的返回值。

```c
// P14
/*
 * classifyAdd3 - classify the exact mathematical sum x+y+z.
 *   Return 1 if the sum is greater than INT_MAX, -1 if it is less than
 *   INT_MIN, and 0 otherwise. You may not use a wider integer type.
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 52
 *   Rating: 7
 */
int classifyAdd3(int x, int y, int z) {
  int low = (x & 3) + (y & 3) + (z & 3);
  int quarter = (x >> 2) + (y >> 2) + (z >> 2) + (low >> 2);
  int bound = 1 << 29;
  int above = ~((quarter + ~bound + 1) >> 31) & 1;
  int below = (quarter + bound) >> 31;
  return above | below;
}
```

**15. floatScaleThreeHalves**

题目要求返回输入浮点数乘以 1.5 后的位编码，并采用选择最近值，中点取偶数的舍入规则。由于不能直接使用浮点运算，通过整数运算处理浮点数的各部分。

首先，我从输入编码中分别取出符号位、阶码和尾数。由于乘以正数 1.5 不会改变符号，保留原符号位。

其次，检查特殊值。阶码的八位全部为 1 时，输入表示无穷大或 NaN，直接返回输入。阶码为 0 时，输入表示零或非规格化数。将尾数乘以 3，再右移

一位，实现除以 2。右移时可能丢失最低的一位：如果丢失位为 1，精确结果恰好位于两个可表示值的中间，保留结果的最低位为 1 时需要加一，使结果的最低位变为 0。

对于普通的数，编码中省略了有效数字最前面的 1，因此先补出这个 1，再将完整有效数字乘以 3。将乘积右移一位，就能得到乘以 1.5 后的有效数字。如果乘积右移一位后超过了 24 位有效数字能够容纳的范围，就把阶码增加 1，并改为右移两位。

随后，我根据右移时舍弃的位完成舍入。舍弃部分小于一半时，保留当前结果；大于一半时，将结果加一；恰好等于一半时，检查保留结果的最低位，最低位为 1 就加一，为 0 就保持不变。

舍入加一后，有效数字可能再次超出原来的位数。因此，检查舍入是否产生进位；如果产生进位，就将有效数字右移一位，并将阶码增加 1。如果调整后的阶码达到无穷大，就返回带有原符号的无穷大。

最后，去掉规格化数无需存储的最高有效位，将调整后的阶码、尾数和原符号位组合成返回编码。具体实现代码如下：

```c
unsigned floatScaleThreeHalves(unsigned uf) {
  unsigned sign = uf & 0x80000000u;
  unsigned exponent = (uf >> 23) & 255;
  unsigned fraction = uf & 0x7fffffu;
  unsigned product, significand, remainder, half, mask;
  int shift = 1;

  if (exponent == 255) {
    return uf;
  }
  if (exponent == 0) {
    product = fraction * 3;
    return sign | ((product >> 1) + ((product & 3) == 3));
  }

  product = (fraction | 0x800000u) * 3;
  if (product >= 0x2000000u) {
    exponent = exponent + 1;
    shift = 2;
  }
  mask = (1u << shift) - 1;
  remainder = product & mask;
  half = 1u << (shift - 1);
  significand = product >> shift;
  if (remainder > half ||
      (remainder == half && (significand & 1))) {
    significand = significand + 1;
  }
  if (significand == 0x1000000u) {
    significand = significand >> 1;
    exponent = exponent + 1;
  }
  if (exponent >= 255) {
    return sign | 0x7f800000u;
  }
  return sign | (exponent << 23) | (significand & 0x7fffffu);
}
```

**16. floatRoundEven**

题目要求将输入浮点数舍入到最近的整数，并返回该整数的单精度浮点位编码。如果输入恰好位于两个整数的中间，就选择偶数。舍入结果为零时需要保留原符号，NaN 和无穷大则保持原编码。

首先，取出输入的符号位和阶码。符号位用于保留结果的正负，阶码用于判断输入的大小，以及哪些位属于整数部分、哪些位属于小数部分。

对于阶码较大的输入，直接返回原编码。单精度浮点数只有 24 位有效数字，当绝对值达到 2 的 23 次方时，可表示的有限数已经没有小数部分，因此无需舍入。NaN 和无穷大的阶码也在此范围内。

对于绝对值小于 0.5 的输入，最近的整数一定是 0，因此只保留原符号位。对于绝对值大于或等于 0.5、但小于 1 的输入，进一步检查尾数。尾数全为 0 表示绝对值恰好为 0.5，返回 0；尾数不为 0 则表示绝对值大于 0.5，返回 1。

对于其余需要处理的输入，根据阶码计算小数部分占据的位数，再构造一个只有这些低位为 1 的掩码。用该掩码可以取出小数部分；将掩码取反后与原编码按位与，则可以清除小数部分，同时保留符号和整数部分。

比较取出的小数部分与代表 0.5 的值。小数部分小于 0.5 时，保留当前整数部分；大于 0.5 时，使整数部分增加 1；恰好等于 0.5 时，检查整数部分的最低位。整数部分为奇数就增加 1，使其变为偶数；整数部分为偶数则保持不变。

```c
unsigned floatRoundEven(unsigned uf) {
  unsigned sign = uf & 0x80000000u;
  unsigned exponent = (uf >> 23) & 255;
  unsigned shift, mask, remainder, half, result;

  if (exponent >= 150) {
    return uf;
  }
  if (exponent < 126) {
    return sign;
  }
  if (exponent == 126) {
    if (uf & 0x7fffffu) {
      return sign | 0x3f800000u;
    }
    return sign;
  }

  shift = 150 - exponent;
  mask = (1u << shift) - 1;
  remainder = uf & mask;
  half = 1u << (shift - 1);
  result = uf & ~mask;
  if (remainder > half ||
      (remainder == half && ((result >> shift) & 1))) {
    result = result + (1u << shift);
  }
  return result;
}
```

**17. float_i2f**

题目要求将有符号整数转换为单精度浮点数，并返回对应的位编码。

首先，单独处理输入为 0 的情况，直接返回正零的编码。对于非零输入，提取原整数的符号位，并将输入的位模式保存在无符号整数中。如果输入为负数，就通过取反加一得到其绝对值；如果输入为正数，则保持不变。使用无符号整数保存绝对值。

接着，确定绝对值中最高的 1 所在的位置。使用一个临时变量保存绝对值，反复将临时变量右移一位，并记录移动次数，直到临时变量只剩下最低位的 1。移动次数就是最高有效位的位置，据此计算阶码。

然后，调整有效数字的位置。单精度浮点数能够保留 24 位有效数字，如果整数的最高有效位不超过第 23 位，就将绝对值左移，使最高的 1 到达第 23 位。如果最高有效位超过第 23 位，就需要将绝对值右移，只保留最高的 24 位。

同时，取出被右移舍弃的低位，用于判断是否需要舍入：将舍弃部分与半个舍入单位比较。舍弃部分小于一半时，保留当前结果；大于一半时，将保留结果加一；恰好等于一半时，检查保留结果的最低位。最低位为 1 就加一，最低位为 0 就保持不变。

舍入加一可能使原来的 24 位有效数字产生第 25 位的进位。如果出现这种情况，我将有效数字再右移一位，并将阶码增加 1，使有效数字重新符合存储范围，同时保持舍入后的数值不变。

最后，我去掉有效数字中无需存储的最高位 1，只保留尾数的 23 位，再将尾数、阶码和原符号位组合成返回编码。实现代码如下：

```
unsigned float_i2f(int x) {
  unsigned sign = x & 0x80000000u;
  unsigned magnitude = x;
  unsigned temp, exponent, significand, remainder, half;
  int highest = 0;
  int shift;

  if (x == 0) {
    return 0;
  }
  if (sign) {
    magnitude = ~magnitude + 1;
  }
  temp = magnitude;
  while (temp >> 1) {
    temp = temp >> 1;
    highest = highest + 1;
  }
  exponent = highest + 127;
  if (highest <= 23) {
    significand = magnitude << (23 - highest);
  } else {
    shift = highest - 23;
    significand = magnitude >> shift;
    remainder = magnitude & ((1u << shift) - 1);
    half = 1u << (shift - 1);
    if (remainder > half ||
        (remainder == half && (significand & 1))) {
      significand = significand + 1;
    }
    if (significand >> 24) {
      significand = significand >> 1;
      exponent = exponent + 1;
    }
  }
  return sign | (exponent << 23) | (significand & 0x7fffffu);
}
```

### 18. bitCount

题目要求统计整数的 32 位二进制表示中共有多少个 1。采用分组相加的方法：先统计每两位中的 1，再合并成每四位、每八位的计数，最后得到全部 32 位的总数。

首先，构造三个掩码，分别用于提取每两位、每四位和每八位中的低半部分。三个掩码为 0x55555555、0x33333333 和 0x0F0F0F0F。从单字节的小常量开始，通过左移和按位或，将相同的位模式扩展到整个整数。

0x55555555 的二进制位交替为 0 和 1，能够取出每组两位中的低位。先提取这些低位，再将原整数右移一位，使用同一个掩码提取并对齐每组的高位。

随后，将相邻两组的计数合并，统计每四位中的 1。使用 0x33333333 提取每个四位组中低两位保存的计数，再将当前结果右移两位，提取高两位保存的计数。

将每个字节中的两个四位组继续合并。把当前结果右移四位后与未移动的结果相加，使每个字节的低四位得到两个小组的计数之和，再用 0x0F0F0F0F 清除高四位中的无关内容，

将当前结果右移八位并与自身相加。相加后，最低字节保存原来低两个字节的计数总和，从低位数起的第三个字节保存原来高两个字节的计数总和。再将当前结果右移十六位并与自身相加，就把这两个总和合并到最低字节，得到全部四个字节中 1 的总数。用低六位为 1 的掩码保留总数，清除更高位中的其他计算结果。

```c
// P18
/*
 * bitCount - return count of number of 1's in the binary representation of x
 *   Examples: bitCount(5) = 2, bitCount(7) = 3
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 40
 *   Rating: 10
 */
int bitCount(int x) {
  int mask1 = 0x55 | (0x55 << 8);
  int mask2 = 0x33 | (0x33 << 8);
  int mask4 = 0x0f | (0x0f << 8);
  mask1 = mask1 | (mask1 << 16);
  mask2 = mask2 | (mask2 << 16);
  mask4 = mask4 | (mask4 << 16);
  x = (x & mask1) + ((x >> 1) & mask1);
  x = (x & mask2) + ((x >> 2) & mask2);
  x = (x + (x >> 4)) & mask4;
  x = x + (x >> 8);
  x = x + (x >> 16);
  return x & 63;
}
```

**19. bitReverse**

题目要求将整数的全部 32 位逆序。采用逐步交换分组的方法，依次交换相邻的 1 位、2 位、4 位、8 位和 16 位分组。

首先，从常量 255 出发，通过左移 16 位和按位或，得到掩码 0x00FF00FF。接着，将该掩码左移四位，再与未移动的掩码异或，得到 0x0F0F0F0F。用同样的方法，得到 0x33333333 和 0x55555555。

然后，使用这些掩码完成前四步交换。每一步都将当前整数分成若干组，并交换各组的低半部分和高半部分。先用对应掩码提取低半部分，再将提取结果左移到高半部分的位置；同时，将当前整数右移，使高半部分进入低半部分的位置，再用掩码清除无关内容。最后，将两部分按位或，得到这一轮的交换结果。右移时可能补入的符号位也会被掩码清除。

第一轮交换每两位中的高位和低位，使每个两位组内部逆序。第二轮交换每四位中的两个两位组，使每个四位组内部逆序。随后，交换每八位中的两个四位组，使每个字节内部逆序；再交换每十六位中的两个字节，使每个十六位组内部逆序。最后，交换整数的高十六位和低十六位，再用低十六位全为 1 的掩码清除符号扩展。两部分合并后即可得到所需结果。

```c
// P19
/*
 * bitReverse - Reverse bits in an 32-bit integer
 *   Examples: bitReverse(0x80000004) = 0x20000001
 *             bitReverse(0x7FFFFFFF) = 0xFFFFFFFE
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 34
 *   Rating: 10
 */
int bitReverse(int x)
{
  int mask8 = 255 | (255 << 16);
  int mask4 = mask8 ^ (mask8 << 4);
  int mask2 = mask4 ^ (mask4 << 2);
  int mask1 = mask2 ^ (mask2 << 1);
  x = ((x >> 1) & mask1) | ((x & mask1) << 1);
  x = ((x >> 2) & mask2) | ((x & mask2) << 2);
  x = ((x >> 4) & mask4) | ((x & mask4) << 4);
  x = ((x >> 8) & mask8) | ((x & mask8) << 8);
  return (x << 16) | ((x >> 16) & (255 | (255 << 8)));
}
```